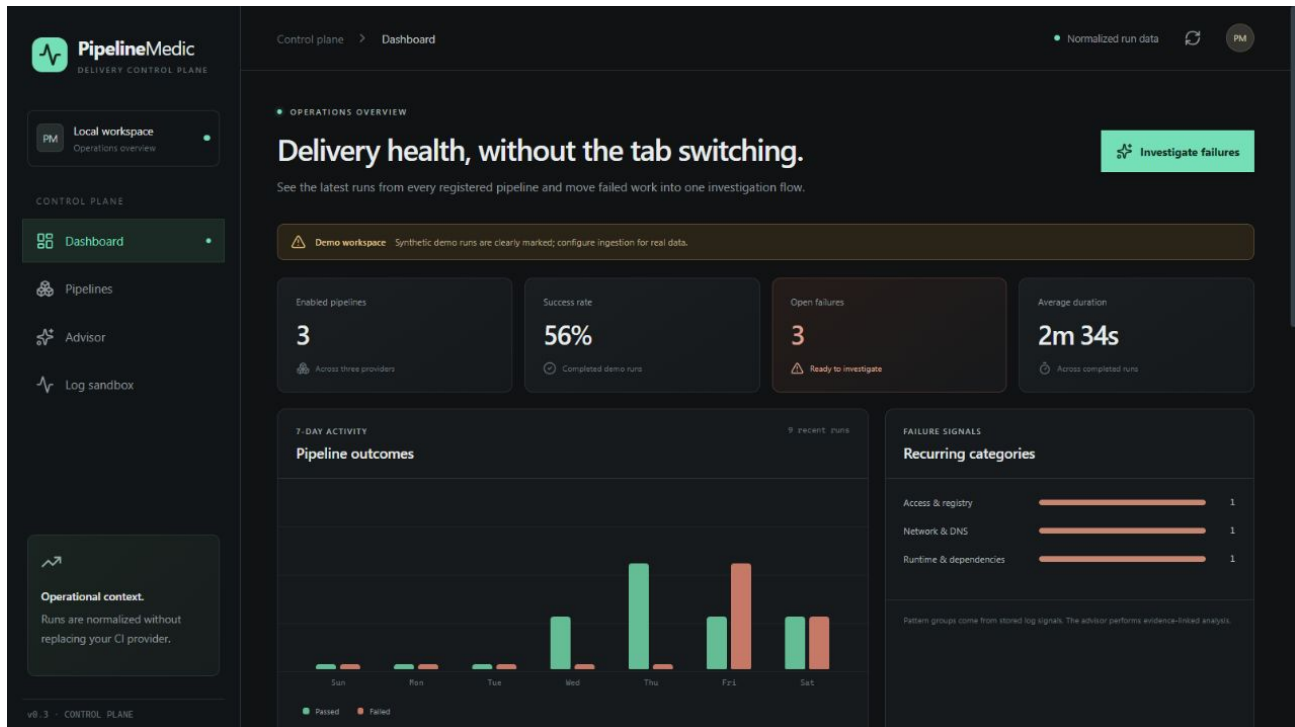


From a failed run to a verified next step

A practical walkthrough for developers, DevOps engineers and hackathon reviewers. Version 0.3 | October 8, 2026. All screenshots show synthetic demo data.



The dashboard gathers cross-provider health and routes failures into one investigation workflow.

Start locally

1. Install Node.js 24 or newer and clone github.com/nguyenphu123/pipelinemedic.
2. In the project folder run `npm ci`, then `npm run dev`. Open `http://127.0.0.1:3000`.
3. Start with the synthetic runs and local rules provider. No API key, CI account or model download is needed for this walkthrough.

Alternative: run `docker compose up --build` and open port 3000. Docker execution has not been validated on the development machine. A shared screenshot may show port 3002; use the port printed by your own server.

Follow the evidence

PipelineMedic
DELIVERY CONTROL PLANE

Local workspace
Evidence advisor

CONTROL PLANE

- Dashboard
- Pipelines
- Advisor**
- Log sandbox

Start with the failed run, then follow the evidence.
The advisor combines the stored job log, pipeline configuration and runtime context without executing repairs.

Failure inbox 3 OPEN

- Platform API** #40 release/2.4 →
- Event Worker #91 feature/billing →
- Customer Web #314 main →

SELECTED FAILURE
Platform API #40
GitLab CI · release/2.4 · a81d9cf

release/2.4 100% 1/2 jobs failed on-premises

Job log | Pipeline config

- Running with gitlab-runner 18.4.0
- Preparing the "docker" executor
- Authenticating with credentials from job payload
- Pulling docker image registry.internal.example/platform/node:24
- Error response from daemon: Head "https://registry.internal.example/v2/platform/node/manifests/24" denied: requested access to the resource is denied**
- ERROR: job failed: exit code 1

Investigate with
Built-in rules Local patterns, Not AI request

Investigate run

Advisor findings RULES

direct evidence

Container registry access was denied
Pattern-based analysis, not AI. Validate these suggestions in your environment.

EVIDENCE-BACKED CAUSE
Container registry access was denied
The registry rejected an image operation. A missing login, insufficient token scope, incorrect repository path or inaccessible repository can produce this response.

Next steps

- Verify the registry hostname and image repository path match the intended project.
- Authenticate in the same job with a credential scoped to the required pull or push operation. Pass passwords using stdin; do not print them.
- Check whether protected variables are available on this branch or merge-request context.

Evidence before advice.
Every displayed citation must point to a real stored log line.

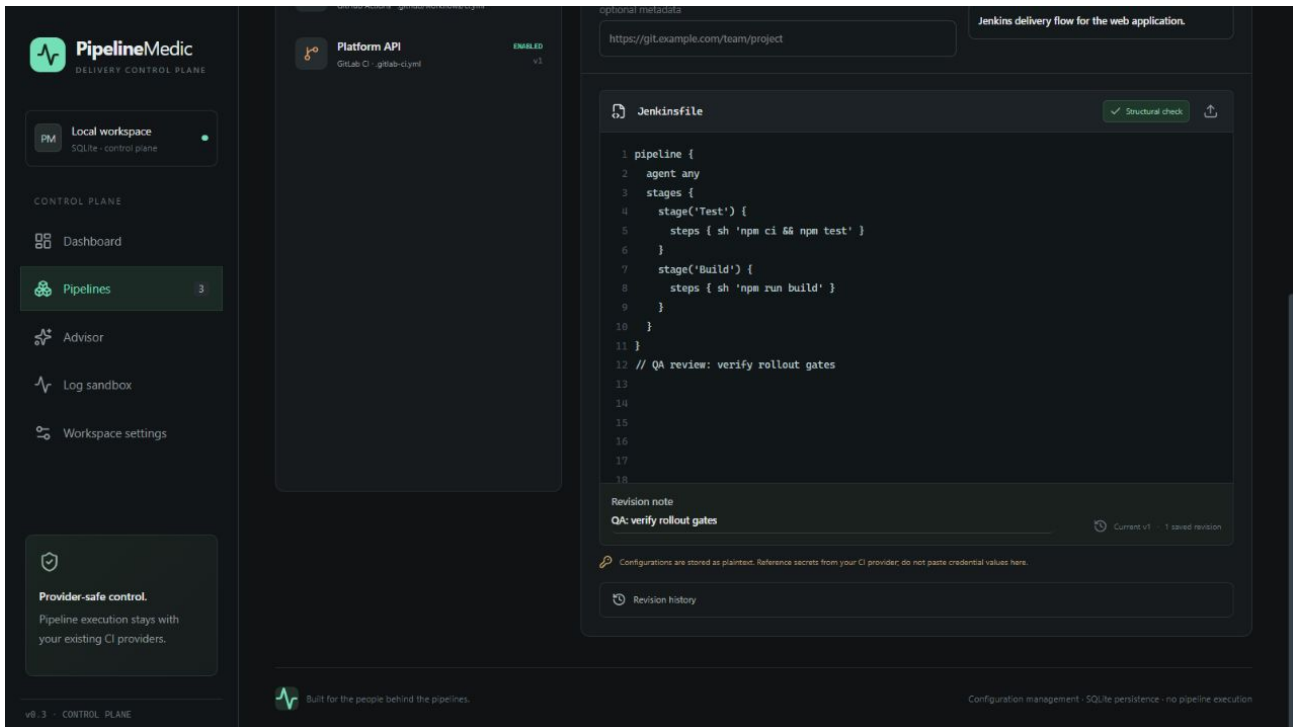
vb. 3 · ADVISOR

A finding citation highlights its supporting log line. Check that line before acting on a recommendation.

1. Open **Dashboard** and use **Investigate failures**, or choose **Advisor** from navigation.
2. Select a failed run from **Failure inbox**. Review the pipeline, branch, commit, duration, runner location and failed-job count.
3. Switch between **Job log** and **Pipeline config** to compare the failed operation with its configuration.
4. Choose the local rules option under **Investigate with** and click **Investigate run**. Read the summary, certainty, causes, next steps and verification checks.
5. Click an evidence citation. The matching numbered log line is highlighted. For the registry-denial example, verify permissions and image destination before changing credentials.

Expected outcome: a grounded diagnosis with supporting lines and checks. Rules mode is deterministic pattern matching, not an AI call. PipelineMedic offers advice; it does not execute repair commands or change infrastructure.

Review and revise configuration

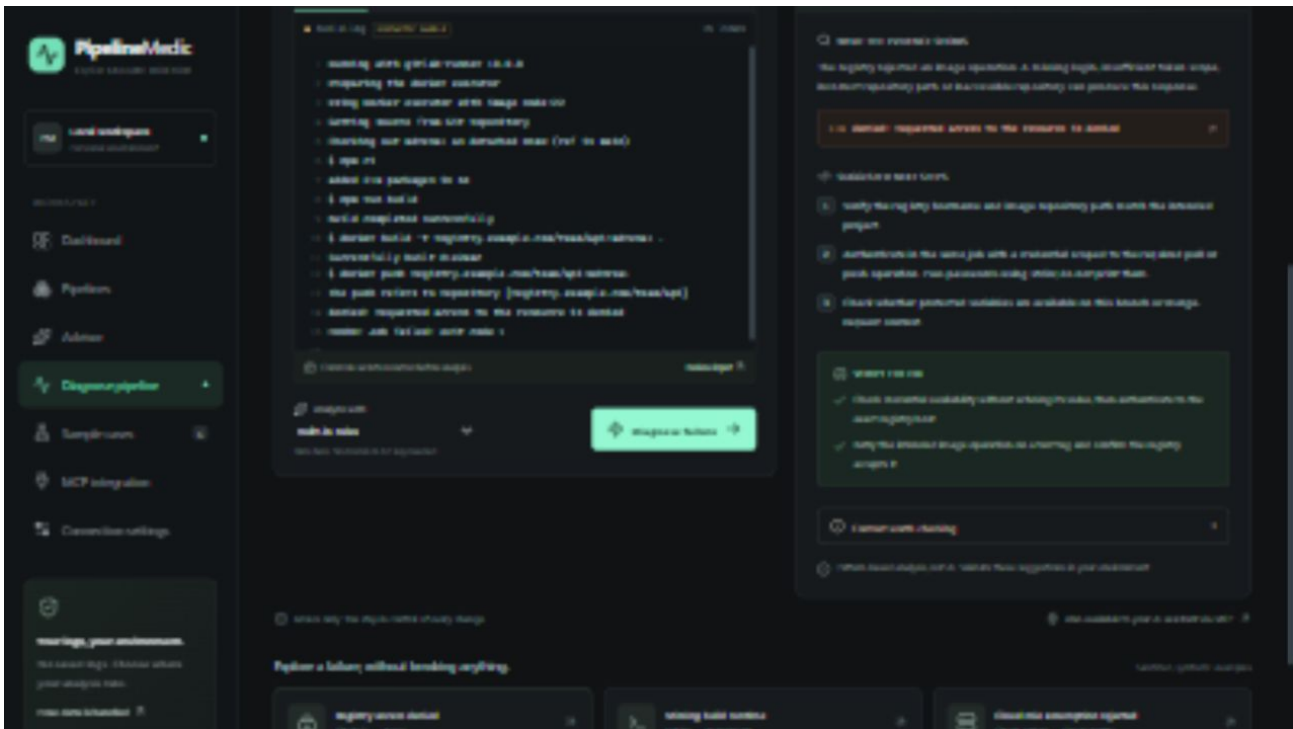


The configuration workbench keeps an editable draft and a local revision trail.

1. Open **Pipelines**, filter or search the catalog, and select a pipeline.
2. Review its provider, configuration path, enabled state and version. Use the editor or upload a supported configuration file.
3. Make a small change, such as a comment explaining a runtime requirement. Add a meaningful revision note and click **Save changes**.
4. Confirm the version increases and inspect the revision history. Download configuration when you need a local copy.
5. Use **Add pipeline** to register another local catalog entry. Enabling or pausing a record changes this catalog; it does not start or pause jobs at the CI provider.

Expected outcome: a saved configuration and revision metadata in local SQLite. Structural checks recognize supported provider shapes; use the provider native linter for full validation. Keep credential values in the CI secret store and reference variables in configuration.

Try an ad hoc failure



The sandbox presents evidence-linked findings and verification steps for pasted or uploaded logs.

1. Open **Log sandbox**. Start with a sample case, or paste/upload a plain-text failed-stage log. Configuration is optional.
2. Set the CI platform, runner location and deployment target independently. Confirm configuration belongs to the current log.
3. Review the redaction preview, choose local rules or a configured model provider, then click **Diagnose failure**.
4. Inspect citations, suggestions and verification steps. Use **Copy report** or **Download report** to export a sanitized Markdown diagnosis.
5. Try the timeout sample. A sparse timeout should request more context rather than assert a particular firewall or networking cause.

Expected outcome: known samples produce evidence-backed findings; ambiguous inputs identify missing evidence. Logs are limited to 100,000 characters / 4,000 lines; configuration to 30,000 characters. Review redaction yourself because it is best effort.

Choose where analysis runs

Cloud or self-hosted AI

1. Copy `.env.example` to `.env` locally. For cloud AI set `CLOUD_BASE_URL`, `CLOUD_API_KEY` and `CLOUD_MODEL`. For Ollama set `OLLAMA_BASE_URL` and `OLLAMA_MODEL`, and install the model separately.
2. Restart the server. Open **Connection settings** in the sandbox to see provider configuration status, then run a diagnosis to test connectivity.
3. Choose the configured provider in Advisor or Log sandbox. Cloud analysis sends sanitized context to that provider; Ollama sends it to the configured model server.

Configured means settings exist, not that connectivity or model quality was tested. No live-model accuracy claim is made for this demo. An unavailable AI provider returns an error; it does not silently substitute rules output. Never put API keys in logs, browser fields or committed files.

Read-only MCP

Configure a stdio-compatible client to launch Node with the absolute path to `mcp/server.js`. See the repository README for the complete JSON configuration and optional environment file. Use these four tools:

`list_pipeline_runs`

List normalized runs, optionally filtered by status.

`diagnose_run`

Analyze a stored run with log and configuration context.

`diagnose_pipeline`

Analyze an ad hoc log and optional configuration.

`list_sample_cases`

Load the six labeled synthetic examples.

If something does not work

Setup needed: configure the provider and restart, or use rules mode. **Unauthorized:** enter the operator-configured workspace access token in Connection settings. **Docker cannot reach Ollama:** use `host.docker.internal` for a host model server. **No grounded finding:** provide the relevant failed stage and matching configuration; keep the uncertainty explicit.

Return to the repository README for setup details, test commands, access controls and known limits. Stored runs and configurations persist in SQLite; sandbox input remains in request/browser state.